

Etude de Cassandra dans le contexte d'un système pour la modélisation du risque tempête

SOMMAIRE

INTRODUCTION	2
PREMIERE PARTIE : PRESENTATION DE CASSANDRA	3
1.1 Introduction.....	3
1.2 Une architecture orientée « cloud ».....	3
1.3 Un modèle de données riche	4
1.4 Un modèle tolérant au partitionnement	5
1.5 Réplication et cohérence « tunable »	5
1.6 Lecture/écriture	6
1.7 Les utilitaires Cassandra	9
SECONDE PARTIE : CHOIX DU SYSTÈME CASSANDRA DANS LE CONTEXTE DE SERIES TEMPORELLES.....	10
2.1 Introduction.....	10
2.2 Un bon modèle pour les séries temporelles	10
2.3 Une bonne scalabilité.....	11
2.4 Une bonne reprise sur panne.....	12
2.4 Le partitionnement et l'ordre sous Cassandra	13
2.5. La limite : difficile compromis entre cohérence et vitesse	14
TROISIEME PARTIE : LA COLLECTE DES DONNEES	15
3.1. Source et description des données.....	15
3.2 Modélisation du Système.	16
3.3 Processus de récupération des données.....	19
CONCLUSION	20
TABLE DES FIGURES	21
PRINCIPAUX LIENS INTERNET	21
BIBLIOGRAPHIE & CONFERENCES	21
ANNEXE – Les scripts.....	22
ANNEXE - Glossaire	29

INTRODUCTION

L'objet de notre étude est de modéliser le risque de tempête en France et d'en calculer l'impact sur un portefeuille d'assurance non-vie à l'aide d'un algorithme. Dans ce contexte, il s'agit de collecter les données météorologiques (vitesse du vent, volume des précipitations, température, etc.) relatives à un certain nombre de points géographiques (stations) susceptibles d'être touchés par une tempête, de simuler les niveaux de vitesse du vent (tempêtes de force 5 6 7 8 9 10 selon l'échelle de Beaufort), les variations de températures, le volume des précipitations, et de désigner des « itinéraires de tempêtes ». Le calcul de l'impact matériel de ces différents niveaux d'intensité de tempête en termes de dommages aux biens (habitations individuelles, véhicules, outils d'exploitation), fera l'objet de la seconde partie de l'étude¹. Afin de calculer cet impact, un portefeuille synthétique d'assurance non-vie sera créé, il sera « stressé » à l'aide d'un générateur de scénarios de tempêtes, permettra une estimation du coût des dommages encourus et proposera les instruments et les tarifs de couverture de ces risques².

Ce rapport s'attache à la collecte massive et quotidienne de données météo à partir d'un site météo, et à déterminer quel est le meilleur choix de système NoSQL qui permettra de stocker et d'exploiter ces données. Il est divisé en trois parties. La première présentera le système NOSQL choisi, en l'occurrence Cassandra. La seconde partie exposera les raisons du choix du système NoSQL pour cette étude au regard du profil des données collectées et des études envisagées. La troisième partie, décrira la mise en place de l'architecture du système choisi. Pour conclure, les limites et les perspectives du système étudié seront exposées.

¹ La seconde partie sera étudiée en RCP216 et STA211.

² Cette partie concernera le projet pour la validation de l'USB03.

PREMIERE PARTIE : PRESENTATION DE CASSANDRA

1.1 Introduction

Cassandra est un système de gestion de base de données (SGBD) NOSQL, un des principaux projets de la fondation Apache. Cassandra est conçue pour gérer des quantités massives de données réparties, en assurant tout particulièrement une disponibilité maximale des données et en éliminant les points individuels de défaillance (SPOF). Initialement développée par Facebook, l'application a été libérée dans l'espace open source et rapidement adoptée par plusieurs grandes entreprises gérant d'importants volumes de données, telles que Twitter, Ebay ou Netflix. On peut décrire Cassandra comme une application NOSQL du type orientée colonne (inspirée du système BigTable de Google) servie par une infrastructure de type clé-valeur (inspirée du système DYNAMO d'Amazon) ; c'est à dire que l'aspect distribué est géré par des ensembles clé-valeur et le stockage par un mécanisme orientée colonne. Dans la suite de cette partie nous allons vous décrire brièvement les principales caractéristiques, sans rentrer dans les détails que vous pourrez facilement trouver sur le web.

1.2 Une architecture orientée « cloud »

Cassandra a été conçue pour assurer une scalabilité horizontale maximale, ou plus simplement elle est faite pour vivre dans un environnement type « cloud de données ». On doit pouvoir ajouter autant de nœuds que l'on souhaite « à chaud ». Elle est basée sur une architecture « fully peer-to-peer », chaque nœud possède les mêmes attributs que les autres. Ils doivent tous être capables de gérer une requête client, de la transmettre aux nœuds possesseurs des données demandées et de répondre au client. La propagation de message a un facteur d'expansion de 3^n , c'est à dire que chaque nœud communique avec 1,2 ou 3 nœuds maximum à la fois, même si chaque nœud connaît la topologie complète du système. Pour assurer cette communication, Cassandra utilise un protocole « peer to peer » nommé Gossip dans une version déterministe (**la communication nœud à nœud n'est pas aléatoire**). Il garantit une très bonne tolérance aux pannes grâce à son mécanisme de « heartbeat³ » toutes les secondes.

La topologie du cluster est totalement configurable, on peut définir des paramètres pour chaque nœud, chaque rack, chaque data center.

Elle est optimisée pour un déploiement sur plusieurs data center éloignés géographiquement, la communication dans ce cas nécessite de désigner des nœuds dits « seeder » pour chaque zone géographique afin d'assurer la communication inter data center (voir figure 1).

³ Système de surveillance par envoi périodique de messages de contrôle appelé « heartbeat ». En cas d'interruption de la communication il est considéré qu'une panne est intervenue.

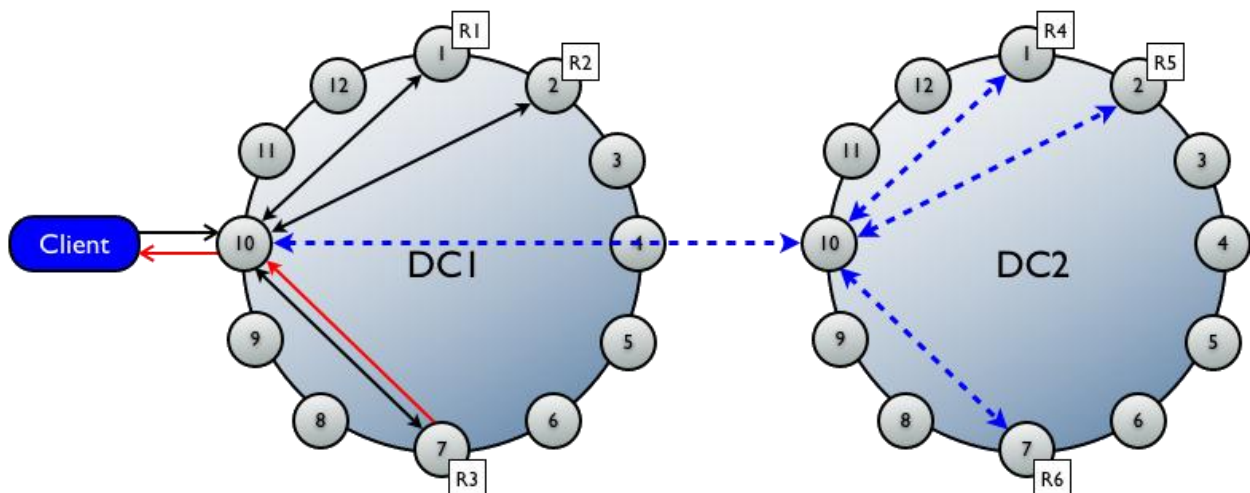


Figure 1 : Un exemple de communication multi data center pour une demande d'écriture. On voit que la cohésion géographique des données est assurée par les seeders (nœud 10 de chaque data center). Le nœud 10 du Data Center 1 joue le rôle de coordinateur puisque c'est lui qui reçoit la requête. Dans une architecture Cassandra multi Data Centers, les seeders jouent d'office le rôle de coordinateur. Il faut en configurer au moins un par Data Center.

1.3 Un modèle de données riche

Sa principale particularité est l'utilisation de deux paradigmes du monde NOSQL assemblés comme des « poupées russes ».

- **Le schéma (ou Keyspace)** : Le stockage dans un système Cassandra se compose de plusieurs schémas de données, équivalent à la base de donnée en relationnel. C'est aussi une structure logique qui n'a pas d'incidence sur l'organisation physique du stockage.
- **La famille de colonne (ou Column Family)** : Chaque schéma possède des familles de colonnes que l'on a tendance à vouloir comparer aux tables relationnelles alors que toute la différence se fait ici (d'ailleurs, ils ont créé un langage client nommé CQL pour faciliter la transition du monde relationnel en reprenant le terme de table). Chaque famille de colonne est une HashMap (objet clé-valeur, qui possède des propriétés différentes d'un tableau) de lignes de colonne. C'est aussi une structure logique qui n'a pas d'incidence sur l'organisation physique du stockage.
- **La ligne (ou row)** : Chaque ligne est une HashMap triée de colonnes. La ligne est la structure qui assure la distribution. La clé du HashMap est unique pour chaque ligne et s'appelle la clé de partition. En effet, Cassandra répartit les données par ligne sur chaque nœud (une ligne ne peut être que sur un seul nœud). C'est cette distribution qui est basé sur le principe clé-valeur de DYNAMO.
- **La colonne (ou Column)** : La colonne est l'unité élémentaire de stockage, c'est un tuple composé d'un nom, d'une valeur et d'un timestamp (qui permet de distinguer les versions de colonne qui possède le même nom). Les clés de colonne (ou clés de regroupement), assurent le tri et la lecture séquentielle d'une ligne sur disque.

Une bonne flexibilité

Il n'est pas nécessaire de savoir quels champs sont présents dans les tables, il est possible d'ajouter et de supprimer des champs « à la volée ». Il s'agit d'un excellent atout pour la productivité, en particulier pour les déploiements de taille non négligeable.

Ce qu'il faut retenir, c'est que l'organisation des données dans Cassandra peut se résumer à cet objet Java (ou visuellement par la figure 2) :

Map<RowKey, SortedMap<ColumnKey, ColumnValue>>

Nous verrons dans la troisième partie l'intérêt de cette structure de données par rapport à la structure en tableau d'une table dans un système relationnel.

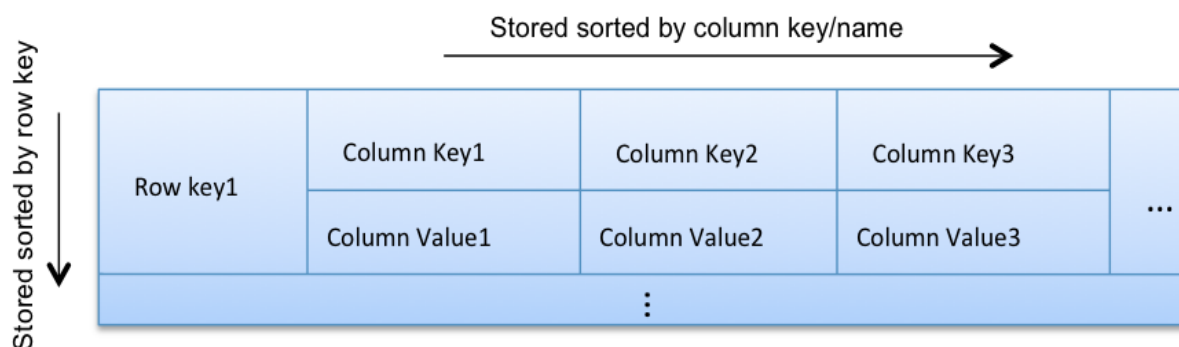


Figure 2 : vue logique d'une ligne dans Cassandra (erratum: les lignes ne sont pas triées comme pourrait le suggérer la partie gauche de la figure).

1.4 Un modèle tolérant au partitionnement

Reprenons le modèle de donnée Cassandra :

Map<RowKey, SortedMap<ColumnKey, ColumnValue>>

Le partitionnement est assuré par la clé de ligne (RowKey). Ainsi les fragments (ou partitions) sont constitués de lignes. Par défaut, Cassandra assure le partitionnement en fragment par le biais d'un algorithme de hachage appliqué à chaque clé de partition. Cela garantit une distribution proportionnelle des fragments sur les nœuds du cluster. Il est aussi possible d'utiliser une fragmentation par intervalles, mais, sauf cas particuliers, cela est fortement déconseillé pour éviter les surcharges de travail sur un serveur (« hot spot »).

1.5 Réplication et cohérence « tunable »

Le partitionnement permet de répartir l'information et son traitement sur différents nœuds, mais ne garantit pas sa disponibilité à tout moment. Pour résoudre ce problème, Cassandra renforce son système par de la réplication. Cette dernière est pilotée par deux piliers : Le facteur de réplication et la stratégie de réplication. Le premier détermine le nombre de copies à conserver pour une donnée ; le second la manière dont sera répliquée l'information entre les nœuds du cluster. Un facteur de réplication RF = 1 est déconseillé car implique qu'une seule copie existera dans le système. Par défaut, le facteur de réplication est fixé à 3 et doit être adapté, avec intelligence, à la taille de la ferme de serveurs. En effet, une valeur trop

élevée peut pénaliser les performances car le système consacrera beaucoup de ses ressources (CPU, RAM) à la réplication et à la synchronisation des données entre nœuds.

Malgré une vitesse d'écriture très supérieure à celle de la lecture, le mécanisme de la réplication induit une certaine incohérence transitoire et cyclique lorsqu'une donnée est répliquée ou mise à jour. Ceci est dû essentiellement au fait que la réplication est un mécanisme asynchrone et dépendant de la vitesse de propagation, donc de la latence réseau. Cette latence peut même devenir gênante si la réplication inter nœuds est réalisée entre « data center » éloignés.

Pour résoudre ce problème, Cassandra – dotée d'un mécanisme de synchronisation nommé « read-repair », permet d'agir sur le niveau de cohérence d'une donnée, on parle de « tunable consistence ». Elle met à disposition du client différents niveaux de cohérence (Zéro, One, ALL, Any, Quorum, etc) qui peuvent être ajustés aussi bien en écriture qu'en lecture. Le niveau « Zéro et Any » sont les niveaux de cohérence les plus faibles ; le niveau « All » est le plus strict et donc le plus impactant du point de vue performance. Ces niveaux sont donc à utiliser avec prudence. Ce n'est qu'une fois la synchronisation achevée, que la cohérence du système est atteinte.

1.6 Lecture/écriture

Écriture

Cassandra est optimisée pour l'écriture. Les anglo-saxons parlent d' « upsert » pour le décrire (tout insert est en fait un update et un update peut faire un insert). D'un point de vue technique, l'écriture repose sur un algorithme nommé Log-structured merge-tree (LSM tree). Comme on peut le voir dans la figure 3, lorsqu'une demande d'écriture est faite, elle est écrite en mémoire sur une structure nommée memtable (et cela est bien entendu vrai pour tous les nœuds répliqués). Le message d'acquiescement de l'écriture se fait juste après. Pour assurer une reprise sur panne, la demande est en même temps sauvée sur un fichier log nommé commit log. Lorsque la memTable est pleine, elle va être sauvée sur disque au format SSTable (un fichier de lignes regroupées et triées). La SSTable est immuable (non-modifiable), lorsque elle est pleine on écrit sur une autre SSTable. Supposons alors que l'on fasse une écriture W pour créer une ligne l: W(l) un jour j. À j+1, cette SSTable est pleine. À j+5, on fait une autre écriture W1 sur cette même ligne W1(l). Cette écriture se fera sur une autre SSTable. Or, il faut se rappeler que Cassandra doit garantir que chaque ligne soit écrite séquentiellement sur disque. Pour résoudre cela, elle utilise un système appelé compaction (compactage en français) qui va fusionner plusieurs SSTable en une tout en triant les données par ligne et colonne.

Maintenant voyons comment Cassandra gère les mises à jour au niveau d'un nœud puisque l'on a vu qu'aucune écriture n'était supprimée. Supposons la famille de colonne suivante pour stocker des enchères :

#CQL

```
Create Table enchere(  
date Timestamp,  
nom_enchere text,  
montant_enchere float,  
PRIMARY KEY(nom_enchere,date))
```

La clé de partition est donc le nom de l'enchère et la clé de regroupement est la date en millisecondes. Un enchérisseur « e » fait une enchère à la date « d » et un autre enchérisseur

fait de même à la même date (même milliseconde) et pour la même enchère. Il y a aura donc deux colonnes créées pour le même nom de colonne. Si je fais une lecture sur cette colonne, quelle valeur vais-je voir ? C'est là que le timestamp de colonne que l'on a vu en 1.3 est utile. Il est écrit en microsecondes. Ainsi Cassandra lira la colonne avec le timestamp le plus récent à la microseconde prêt. On peut aussi faire une requête en posant une condition sur la valeur de ce timestamp si l'on veut récupérer une version particulière de cette colonne.

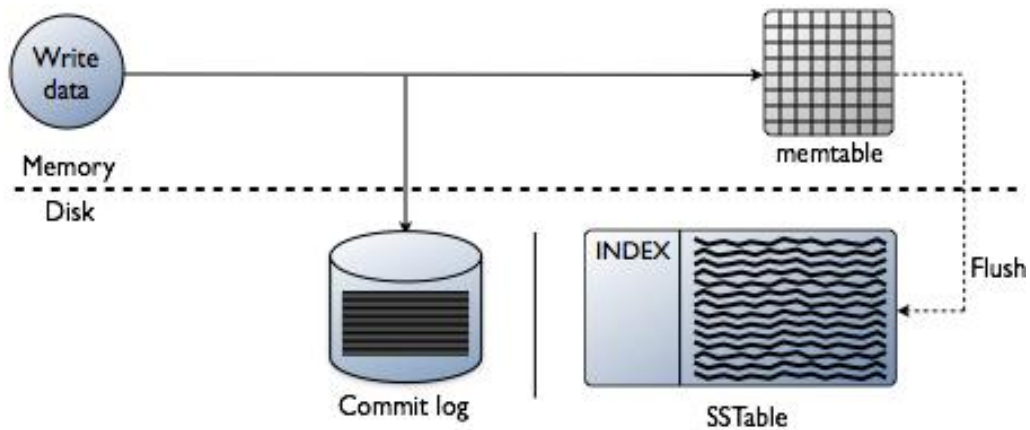


Figure 3 : vue du mécanisme d'écriture dans Cassandra.

Lecture

Ce mécanisme d'écriture a une influence sur le mécanisme de lecture. En effet, si l'on veut faire une lecture R sur une ligne l $R(l)$, on peut très bien avoir une partie des colonnes se trouvant encore en mémoire, et une ou plusieurs sur disque dans des SSTable.

Prenons l'exemple de la figure 4, avec un cluster sur 3 Data Center pour une requête read R toute simple sur une ligne l pour une seule colonne c : $R(l,c)$. On a un schéma (Keyspace) avec un facteur de réplcation de 5 et un niveau de cohérence (consistency level) de 3, c'est à dire que pour acquitter une lecture, au moins 3 nœuds possédant la colonne recherchée doivent répondre.

1. Le client envoie sa requête $R(l,c)$ au système. Il est connecté au nœud 9 (ce qui implique qu'il est seed puisque l'on est sur une architecture multi Data Center).
2. La requête est gérée par le storage proxy du coordinateur. Celui-ci va d'abord vérifier dans la table de hachage (tous les nœuds possèdent la table de hachage du cluster entier) quels sont les nœuds correspondant à la clé de partition de cette ligne. Si le nombre de nœuds trouvé ne peut satisfaire le niveau de cohérence demandé en lecture (en l'occurrence ≥ 3), il envoie un message d'erreur au client. Sinon il passe à l'étape suivante.
3. Le storage proxy a trouvé 5 nœuds possédant des versions de cette ligne. Avec un niveau de cohérence à 3, il va envoyer la requête aux trois nœuds les plus proches (d'un point de vue latence réseau) de lui. Pour le plus proches des trois (en l'occurrence le nœud 2), ce sera une requête read, pour les deux autres, ce sera une requête dite get Diggest : ils ne vont pas renvoyer l'intégralité de la colonne, mais une version hachée de

cette dernière pour pouvoir faire une comparaison entre les versions de colonne de ces trois nœuds.

4. Que ce soit en read ou get diggest, les trois nœuds vont réaliser les mêmes opérations pour trouver la colonne. Ils vont d'abord inspecter leur memTable, le moyen le plus rapide de trouver la donnée puisque qu'elle est unique et en mémoire.
5. Si la colonne concernée n'est pas en mémoire, il va scruter les SSTable sur disque. Tout d'abord le bloom filter de chacune. C'est une fonction probabiliste capable de déterminer pour une clé de partition de donnée la probabilité d'avoir une partie de cette ligne sur la SSTable.
6. Si le bloom filter est positif, le cache d'index de la SSTable va être inspecté pour trouver l'adresse physique de la dernière version de la ligne.
7. Lorsque chacun des trois nœuds a fini son traitement, il envoie la réponse au nœud coordinateur. Celui renvoie la réponse du nœud 2 (le niveau 3 de cohérence n'impose pas d'élections pour choisir la version à envoyer entre les 3 nœuds, il envoie automatiquement la version du nœud le plus proche). Si le mode read-repair est actif, le coordinateur va déterminer la version la plus récente de cette colonne à partir du read du nœud 2 et des diggest des nœuds 3 et 5. Il fera ensuite demande d'écriture pour les nœuds qui ne sont pas à jour. Il fera finalement de même pour les nœuds qui n'ont pas été contactés afin de garantir la cohérence de données sur tous les réplicas.

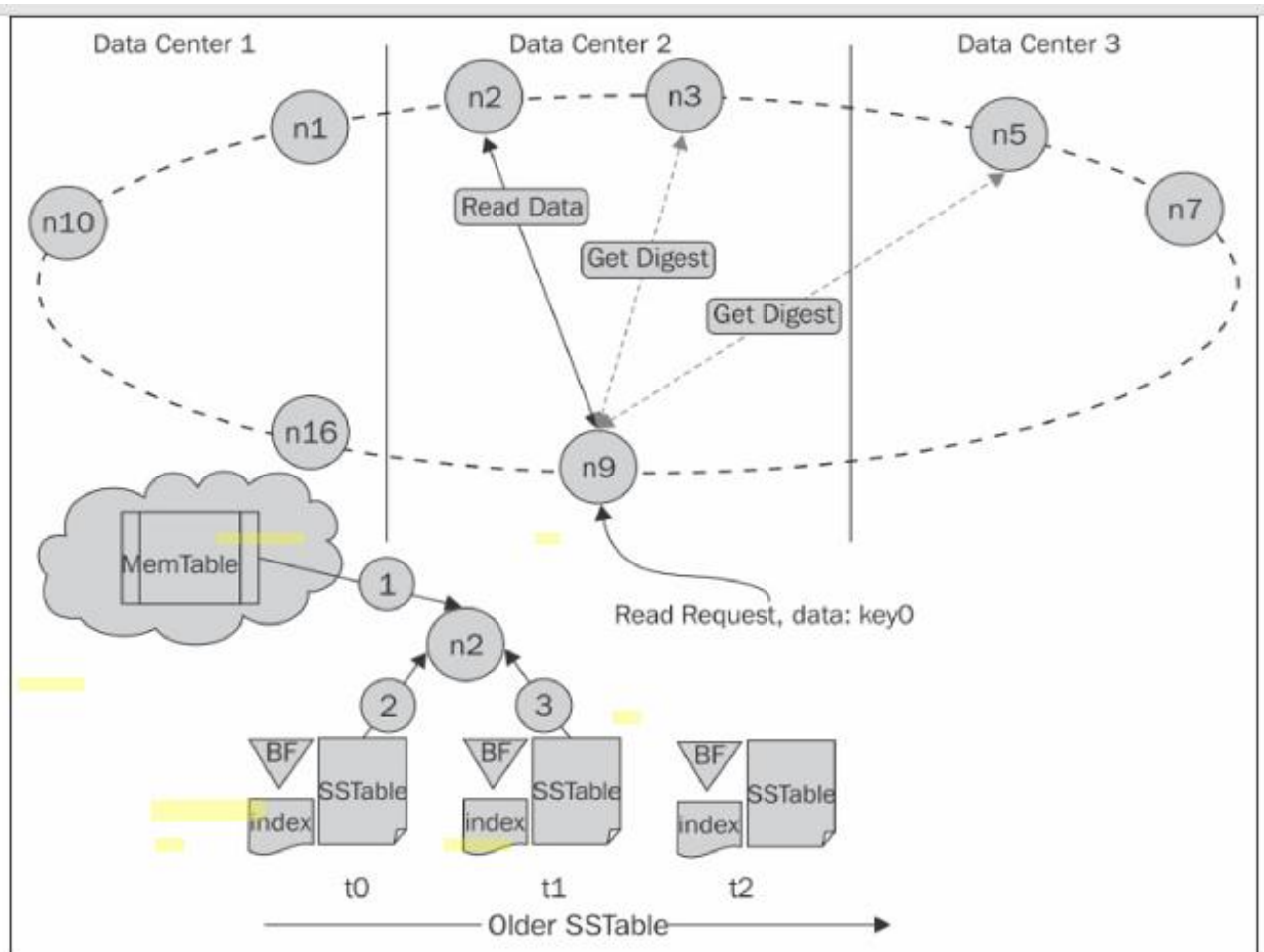


Figure 4 : vue du mécanisme de lecture.

1.7 Les utilitaires Cassandra

De base, Cassandra possède plusieurs utilitaires bien pratiques :

- **CQL** : un langage basé sur le standard SQL qui facilite grandement l'écriture des requêtes. Il abstrait une bonne partie de l'architecture du système pour le client.
- **NODETOOL** : un utilitaire pour inspecter la topologie réseau du cluster.
- Plusieurs outils pour faire de l'écriture en batch à partir de sources de données JSON ou CSV, afin d'optimiser le temps d'utilisation des disques.

D'autre part, la société Datastax spécialisée dans la commercialisation de solutions autour de Cassandra, a mis à disposition de la communauté plusieurs outils très professionnels tels que Devcenter (pour la création de requête CQL), OpsCenter (pour l'administration des clusters), etc. Elle a édité des drivers pour la plupart des langages courants (Java, Python, Ruby, C++, PHP, etc.).

SECONDE PARTIE : CHOIX DU SYSTÈME CASSANDRA DANS LE CONTEXTE DE SERIES TEMPORELLES

2.1 Introduction

Le choix du système NoSQL, avant de se lancer dans toute étude est primordial même s'il est très difficile de trouver le système idéal sans connaître tous les systèmes existants. Pour nous un moteur de stockage de série temporelles efficace, scalable, ayant une bonne reprise sur panne, un modèle de donnée adapté, ainsi qu'un système qui accepte une diversité de formats (xml, json et csv) sont les principaux critères qui nous guident dans le choix du système. Le système de gestion de base de données doit permettre de collecter massivement et quotidiennement des données météo à partir de différentes stations météo.

Il doit donc présenter les qualités suivantes :

- il doit stocker efficacement des données ;
- il doit les stocker séquentiellement ;
- et l'application doit pouvoir les lire séquentiellement.

Il faut pouvoir disposer :

- de toutes les données d'une station météo ;
- d'une seule donnée à une date et une heure unique ;
- d'une série de données pour un temps donné.

La base de données doit supporter les requêtes suivantes :

- stocker la donnée par station météo ;
- stocker des séries temporelles dans l'ordre (du premier événement au dernier événement).

Les raisons qui ont conduit au choix de Cassandra

Parmi tous les systèmes NoSQL à notre disposition, nous avons choisi Cassandra essentiellement parce que c'est un modèle de données orienté colonnes qui accepte les formats xml, json et csv et qui possède un mécanisme de persistance performant.

De plus, le but de l'architecture Cassandra est de garantir une haute disponibilité via une grande tolérance aux pannes doublée d'une extensibilité (scalabilité) facile et flexible, le tout avec une promesse de hautes performances aussi bien à l'écriture qu'à la lecture, ce qui pour notre étude nous semble des qualités importantes puisque l'objectif sera d'exporter notre base sous SAS Enterprise Miner qui est un logiciel statistique orienté colonne et au format « .csv ».

2.2 Un bon modèle pour les séries temporelles

Une série de données temporelles est une série de données ordonnées dans le temps. Dans la plupart des cas, on parle de séries de métriques. En effet, que ce soit des données météorologiques, des benchmarks, des logs, ou des portefeuilles d'actions, on a une ou

plusieurs ressources (dans notre cas des stations météorologiques), une série de relevés dans le temps (on parle de point de données) par ressources, de une ou plusieurs métriques par ressource (température, vitesse du vent, etc.) pour laquelle on a une valeur (voir figure 5). On a donc besoin d'un modèle de type One-To-Many imbriqué.

C'est un modèle qui demande des possibilités d'organisation complexe, et c'est ce qu'offre le modèle orienté colonnes par rapport au modèle clé-valeur qui nous obligera à bricoler avec des collections pour avoir ce résultat.

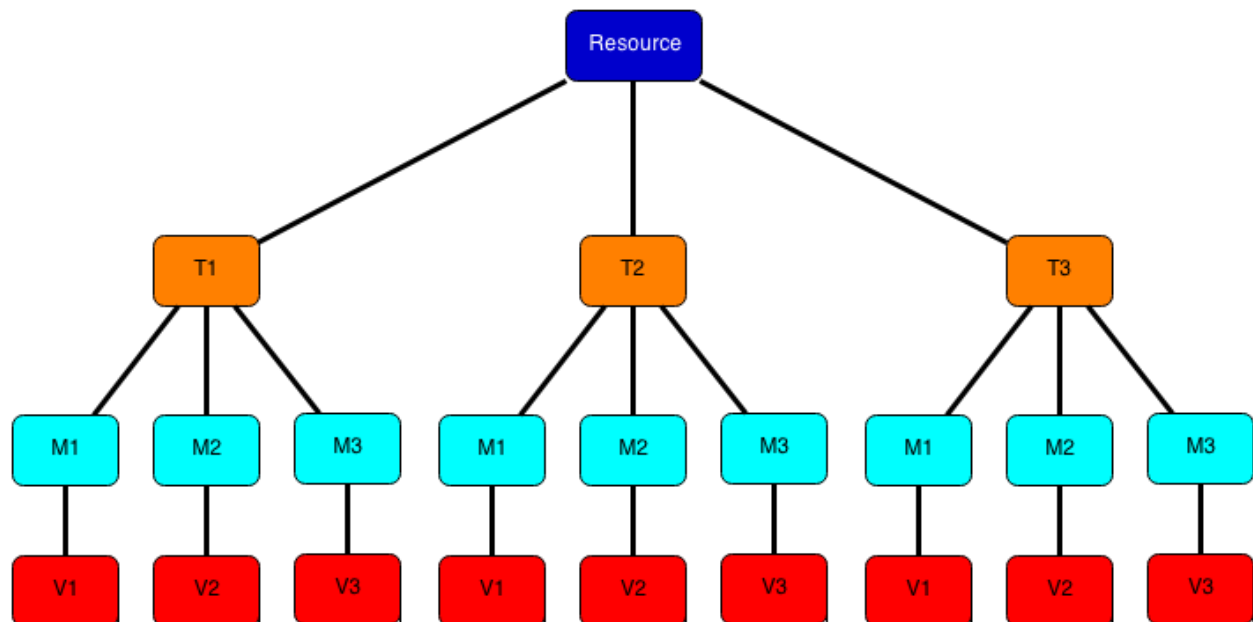


Figure 5 : modèle de données classique pour les séries temporelles

2.3 Une bonne scalabilité

On voit au final que les séries temporelles se modélisent avec des schémas fortement relationnels. Au fond, pourquoi ne pas choisir un bon vieux SGBDR ?

Le point important dans le choix d'un SGBD NOSQL plutôt qu'un SGBDR dans notre cas n'est pas le type de données, mais la scalabilité horizontale des systèmes NOSQL.

Cela fonctionnera très bien sur un serveur pour une centaine de Go. Maintenant si on a 20 To de données, il sera nécessaire de distribuer le système. C'est très complexe à faire avec des SGBDR open source, et les solutions payantes sont très chères et optimiser pour des serveurs à haute disponibilité eux aussi très chers.

La scalabilité de la plateforme Cassandra est telle qu'il suffit d'augmenter progressivement la taille de la ferme de serveurs pour pouvoir absorber plus de charge, ceci avec l'avantage de ne rien modifier à l'architecture applicative en place. Pour remplir ce cahier des charges, Cassandra utilise différentes techniques vues dans la première partie, parmi lesquelles :

- **La communication inter nœuds Peer-to-peer grâce au protocole GOSSIP.**
- **Les filtres « Bloom filter ».**
- **Le « read/repair » et la compaction.**
- **Le partitionnement et la réplication.**

2.4 Une bonne reprise sur panne

Quand on parle de séries temporelles, on parle de systèmes fonctionnant souvent en continue dans le temps. La reprise sur panne est un élément hautement critique pour choisir un système dans notre cas.

Les données d'un nœud sont automatiquement répliquées vers d'autres nœuds. Ainsi, si un nœud est hors service les données présentes sont disponibles à travers d'autres nœuds. Le terme de facteur de réplication désigne le nombre de nœuds où la donnée est répliquée. Par ailleurs, l'architecture de Cassandra définit le terme de cluster comme étant un groupe d'au moins deux nœuds et d'un data center comme étant des clusters délocalisés. Cassandra permet d'assurer la réplication à travers différents data center. Les nœuds qui sont tombés peuvent être remplacés sans indisponibilité du service.

Depuis la version 1.2, Cassandra est doté d'un système de « nœuds virtuels » dit Vnodes, technique qui existait déjà dans d'autres SGBD tels que RIAK, qui optimise grandement la réplication. Sur l'illustration de la figure 6, sans la fonctionnalité des nœuds virtuels (schéma du haut), chacun des 6 nœuds est responsable d'un segment (« range ») des données, ou plus précisément 3 en supposant un facteur de réplication de 3, les trois formant un segment contigu de l'anneau. Avec les Vnodes (schéma du bas), chacun des 6 nœuds est responsable de 2 ou 3 segments (**configurable, par nœud**), qui après réplication font que tous les nœuds ont les données de 8 segments de l'anneau, mais le tout ne forme pas un segment contigu contrairement à précédemment.

En bonus, la configuration du nombre de segments par nœud fait qu'il est possible de définir des nœuds avec plus ou moins de données ce qui est utile en cas de caractéristiques système différentes entre serveurs physiques.

Ajouter un nœud dans le cluster est également beaucoup plus rapide, car désormais davantage de nœuds du cluster peuvent envoyer les données nécessaires à l'initialisation du nouveau nœud.

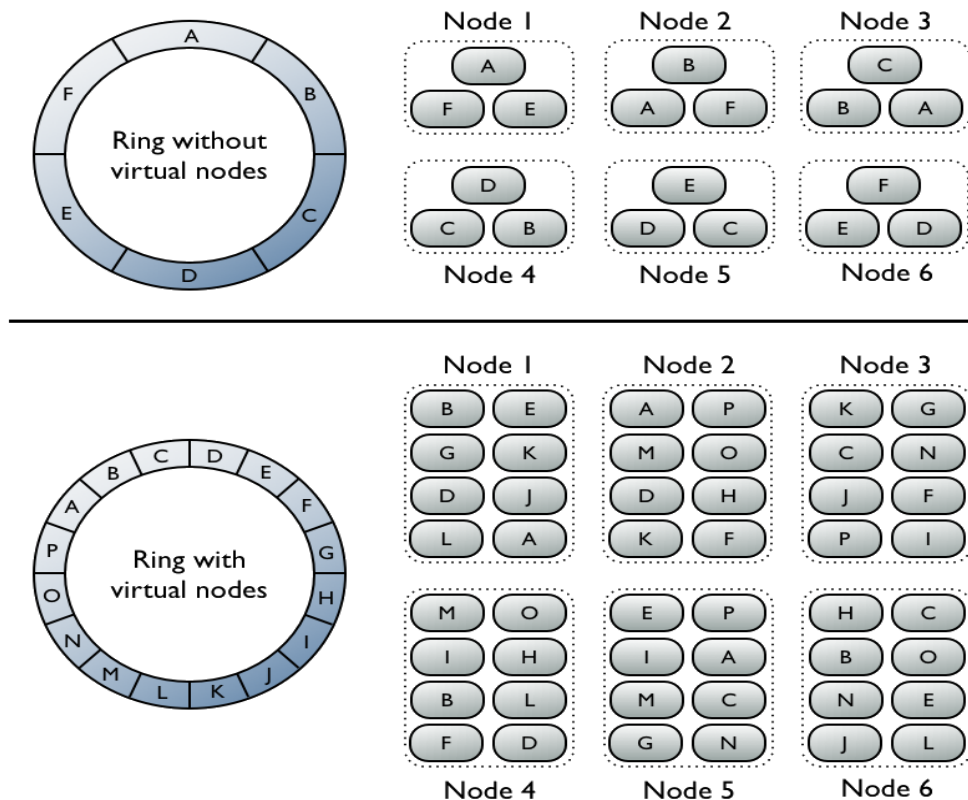


Figure 6 : la différence entre la réplication par noeud de celle par noeud virtuel pour un facteur de réplication à 3

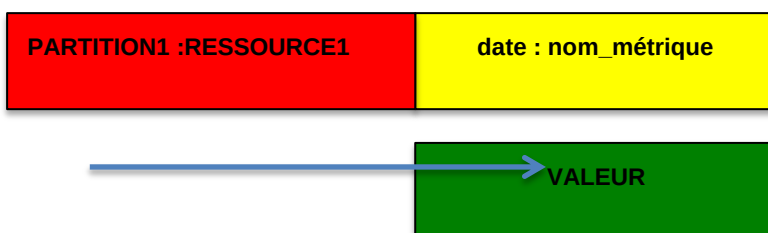
2.4 Le partitionnement et l'ordre sous Cassandra

Une famille de colonne générique pour stocker les points de données d'une série temporelle pourrait être implémentée ainsi en CQL :

```
CREATE TABLE donnees_temp
(
  partition int, // intervalle de temps fixe en secondes (par exemple 1 jour ou un mois)
  ressource text, // dans notre cas des stations météorologiques
  date timestamp, // date au format UNIX de la mesure
  nom_métrique text, // nom de la métrique ( par exemple la vitesse du vent)
  valeur blob, // valeur de la métrique
  PRIMARY KEY((partition, ressource), date, nom_métrique)
);
```

La clé de partition est composée de la partition de temps et de la ressource. Ce qui signifie que toutes requêtes par station(s) météo(s) et/ou intervalles de temps exploiteraient au maximum la distribution de notre cluster.

En ce qui concerne le stockage physique sur les nœuds, prenons l'exemple de la ligne d'index i (partition 1, ressource 1). Physiquement, on pourrait représenter son stockage ainsi :



Colonnes triées et groupées par date, puis triées et groupées par nom de métrique

Tous les éléments dans la partition seront ainsi ordonnés selon la date et l'heure de l'événement puis par métrique. Le partitionnement se fera dans l'ordre de l'événement. Cassandra permet ainsi de disposer des données dans l'ordre car elles sont stockées séquentiellement sur le disque dur. Le moteur de stockage fusionne les données par séquence.

2.5. La limite : difficile compromis entre cohérence et vitesse

Si la cohérence est configurable dans Cassandra, cela ne veut pas dire pour autant que le type de cohérence n'aura pas d'impacts sur le système. Notre système a pour but de faire de l'analyse « temps-réel ». Dans ce genre de cas, il existe une adaptation du théorème de CAP, appelé SCV pour SPEED-CONSISTENCY-DATA VOLUME (voir figure 7). De nombreux benchmarks réalisés par des sociétés comme LinkedIn ont montré que Cassandra ralentissait fortement avec une cohérence de niveau QUORUM. Cela parce que son architecture peer-to-peer obligeait les nœuds répliqués à de nombreux échanges pour voter, faisant croître la latence exponentiellement à partir d'un moment, à la différence d'un système maître-esclave où les élections sont centralisées. Facebook a préféré abandonner Cassandra pour cette raison et préférer HBASE, car le besoin de cohérence pour son système de messagerie était élevé.

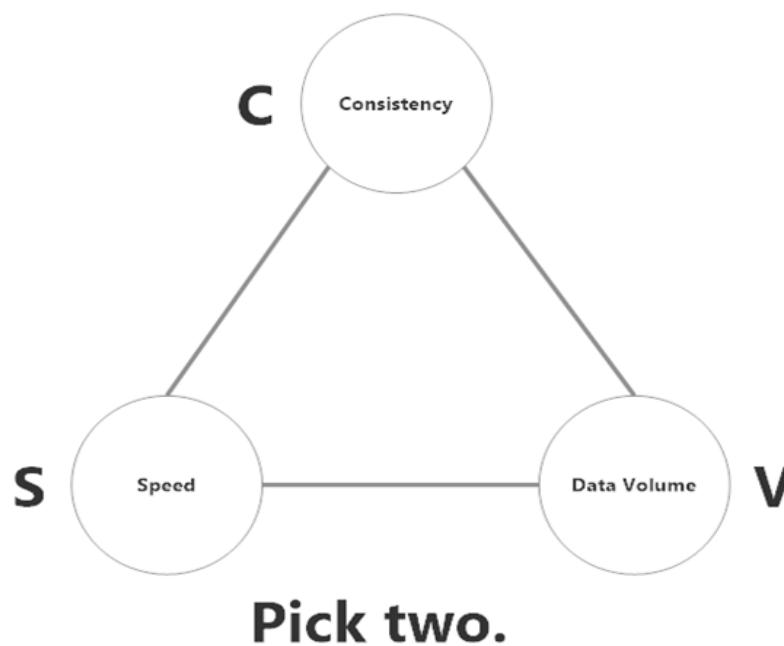


Figure 7: le théorème de CAP adapté aux systèmes analytiques temps-réels.

TROISIEME PARTIE : LA COLLECTE DES DONNEES

3.1. Source et description des données

Source

Les données sont récupérées quotidiennement sur le site OpenWeatherMap (OWM), basé sur le très célèbre « OpenStreetMap », qui offre des données météo d'un pays, d'une région ou d'une ville. À cet accès, s'ajoute une API libre de type http REST donnant accès aux données de météo actuelles, prévisions et cartes avec les phénomènes météo comme les nuages, le vent, la pression et la précipitation. Grâce à cette API, il est possible de collecter les données météo de la région que l'on souhaite aux formats json, xml ou html. En tant que source de données, Openweathermap utilise les services météorologiques officiels de diffusion, des données brutes à partir des aéroports des stations météorologiques, des données brutes de radars, et les données brutes provenant de stations météorologiques officielles.

Recherche : OWM est un système de géocodage qui permet de trouver des villes par nom, pays, code postal ou coordonnées géographiques. Il est également possible de faire des recherches sur une partie du nom de la ville ou de la région géographique.

Données météo actuelles : Données disponibles sur plus de 40 000 villes et plus de 200 000 stations météo à travers le monde, actualisées toutes les 10 minutes.

Prévision : Des prévisions entre 5 et 16 jours sont disponibles pour n'importe quelle localisation. Les données prévisionnelles à 5 jours sont mises à jour toutes les 3 heures et celles à 16 jours une fois par jour.

Description des données

OWM met à disposition plusieurs types de données : des données de météo actuelles, des prévisions de 5 à 16 jours (comme nous venons de l'indiquer plus haut) et également un historique d'un mois de données météo.

Les données disponibles sont les suivantes :

- Param = « temp » (température)
- = « pressure » (pression)
- = « humidity » (humidité)
- = « wind_speed » (vitesse du vent, mètre par seconde)
- = « rain » (pluie)
- = « deg » (direction du vent)

Ces données nous fourniront assez d'éléments pour caractériser les vitesses de vent par régions qui nous intéressent.

Exemple d'extraction de donnée :

```
{"coord":{"lon":-1.68,"lat":48.11},"sys":{"message":0.1759,"country":"France","sunrise":1424329546,"sunset":1424367316},"weather":[{"id":802,"main":"Clouds","description":"scattered clouds","icon":"03d"}],"base":"cmc stations","main":{"temp":282.349,"temp_min":282.349,"temp_max":282.349,"pressure":1029.68,"sea_level":1038.26,"grnd_level":1029.68,"humidity":76},"wind":{"speed":7.12,"deg":218.502},"clouds":{"all":32,"dt":1424360664,"id":6432801,"name":"","cod":200}}
```

Dans notre exemple :

Longitude -1.68 et latitude 48.11 correspondent à la ville de Rennes.

La vitesse du vent est égale à 7,12 mps.

La vitesse est donnée en « mps » (c'est-à-dire en mètre par seconde), nous utiliserons la table de Beaufort pour convertir si besoin les données en km/h et pour qualifier les vents :

Beaufort	Description	km/h	m/s	knots (nœuds)	Etat des eaux
0	Calme	< 1	<0.5	<1	Comme un miroir
1	Très légère brise	1.9 - 6.4	0.5 - 1.8	1 - 3	Quelques rides sur l'eau
2	Légère brise	6.5 - 12.0	1.9 - 3.3	4 - 6	Vaguelettes ne déferlant pas
3	Petite brise	12.1 - 19.4	3.4 - 5.4	7 - 10	Les moutons apparaissent
4	Brise	19.5 - 28.7	5.5 - 7.9	11 - 15	Petites vagues, nombreux moutons
5	Bonne brise	28.8 - 39.8	8.0 - 11.0	16 - 21	Vagues modérées, moutons, embruns
6	Forte brise	39.9 - 50.9	11.1 - 14.1	22 - 27	Lames, crêtes d'écume blanche, embruns
7	Grand frais	51.0 - 62.0	14.2 - 17.2	28 - 33	Lames déferlantes, traînées d'écume
8	Coup de vent	62.1 - 75.0	17.3 - 20.8	34 - 40	Tourbillons d'écume à la crête des lames, traînées d'écume
9	Fort coup de vent	75.1 - 87.9	20.9 - 24.4	41 - 47	Lames déferlantes grosses à énormes, visibilité réduite par les embruns
10	Tempête	88.0 - 102.8	24.5 - 28.5	48 - 55	
11	Violente tempête	102.9 - 117.6	28.6 - 32.6	56 - 63	
12	Ouragan	> 117.6	> 32.6	> 63	

Figure 8 : Échelle de Beaufort.

Selon l'échelle de Beaufort ci-dessus, 7.12 m/s correspond à une force 4 (brise).

3.2 Modélisation du Système.

On va supposer que l'on possède une infrastructure de 100 nœuds répartis sur 25 serveurs physiques avec 10To par serveur de disques et 32Go de mémoire RAM par serveur. Cela nous donne 8Go par nœud ce qui est optimal pour la machine virtuelle Java de Cassandra d'après les spécifications de la fondation Apache. Nous voulons créer notre base avec un facteur de réplification de 3.

Il s'agit dans un premier temps de nous bâtir une base d'apprentissage en récupérant les données météorologiques françaises sur une période continue avec la fréquence maximale que notre système peut accueillir.

Pour cela, nous devons modéliser un schéma (ou keyspace) Cassandra. Si on reprend le modèle de schéma pour les séries temporelles vu en 2.3, on aura une seule famille de colonnes générée par le code CQL ci-dessous :

CREATE TABLE donnees_temp

```
(
  partition long, // intervalle de temps fixe en secondes (par exemple 1 jour ou un
mois)
  station_id int, // dans notre cas des stations météorologiques
  collected_at timestamp, // date au format UNIX de la mesure
  metric_name text, //nom de la métrique ( par exemple la vitesse du vent)
  val blob, // valeur de la métrique au format binaire
  PRIMARY KEY((partition, station_id ), collected_at, metric_name)
);
```

Si on stocke 100 jours partitionnés en intervalles heures de données, tout en exploitant la fréquence maximale fournie par OpenWeatherMap, en l'occurrence 1 point toutes les millisecondes, pour les 4 282 stations en France, ainsi que les 24 métriques fournies par point, cela va nous donner une quantité en milliers de To. Nous nous contenterons d'un point de données toutes les secondes.

Cela nous donne le tableau de dimensionnement suivant :

MODELISATION 1	Modélisation avec une fréquence de 1 point de données par secondes	
NR	Nombre de lignes: 4282*100*24	10 276 800
NC (per row)		86 400
CNS	Nombre de colonnes par ligne: : 60*60*24 métriques	8
CVS	taille moyenne du nom d'une colonne (octets) taille moyenne de la valeur d'un colonne (octets): (18*4+10+10+10+30+8+8)*24	6
RKS	taille d'une clé de ligne (octets)	8
RF	nombre de réplication	3
Number of Nodes	nombre de nœuds	100
IPS		64
TNC (total)		
	nombre total de colonne=NR X NC	887 915 520 000
Column data	quantité de données totale des valeur de colonnes (en octet) =CVS X TNC	5 475 479 040 000
Column overhead	surcoût lié à la génération de colonnes CNS X TNC	20 422 056 960 000
Column Subtotal	Data+Column overhaed	25 897 536 000 000
Row header overhead	surcoût lié à la génération de lignes(octet)	318 580 800
Row bloom filter		444 091 358 400
Row index		14 567 405 107 200
Row Subtotal	total lignes	15 011 815 046 400
Data file subtotal	Column Subtotal+Row Subtotal	40 909 351 046 400
Per Node		1 227 280 531 392
SSTABLE index		184 982 400
SSTABLE bloom filter		19 269 003
Base Storage	Data file subtotal	40 909 555 297 803
Additional Replicas	réplicas additionnels	81 819 110 595 605
Compaction Overhead	surcoût additionnel dû au compactage	122 728 665 893 407
Total Storage	STOCKAGE TOTAL	245 457 331 786 815
Per Node	Total par neoud	2 454 573 317 868
Actual column value data	% pris par les données en colonnes	13,38%
Column overhead	% du surcoût de la génération de colonnes	49,92%
Row overhead	% du surcoût de la génération de lignes	36,70%
SSTABLE overhead	% surcoût additionnel dû au compactage	0,00%

Figure 9 : Dimensionnement du schéma, premier test.

On constate que notre infrastructure est suffisante pour le stockage de 100 jours avec un point de données toutes les secondes. Cela nous donne un peu moins 2,5 To par nœud. D'autant que nous n'avons pas pris en compte la fonction de compression de Cassandra qui peut réduire la taille des données avec un facteur allant de 2 à 4.

Toutefois ce n'est pas satisfaisant. On remarque en bas du tableau que le pourcentage du stockage total pris par nos données rentrées n'est que de 13,38%. Cela est dû au fait que Cassandra génère un nombre conséquent de métadonnées par colonne. Il faut donc que l'on optimise cela en modifiant notre schéma. Au lieu de faire une colonne par métrique, nous allons stocker nos métriques par point de données et par station dans une seule colonne au format JSON. On a maintenant le dimensionnement suivant.

On remarque que nos données métiers représentent maintenant plus de 55% du total stockés. D'autre part, le stockage par nœud est passé de 2,5 To à 594 Go. On a quasiment économisé de 2To par nœud de métadonnées ! L'inconvénient est que l'on perd en qualité de recherche d'information, mais cela ne nous gêne pas, car nous avons l'intention de prétraiter les données avec Apache SPARK.

MODELISATION 2		Modélisation avec une fréquence de 1 point de données par secondes	
NR	Nombre de lignes: 4282*100*24		10 276 800
NC (per row)			3 600
CNS	Nombre de colonnes par ligne: : 60*60*24 métriques		8
CVS	taille moyenne du nom d'une colonne (octets) taille moyenne de la valeur d'un colonne (octets): (18*4+10+10+10+30+8+8)/24		148
RKS	taille d'une clé de ligne (octets)		8
RF	nombre de réplacaiton		3
Number of Nodes	nombre de nœuds		100
IPS			64
TNC (total)			
Column data	nombre total de colonne=NR X NC quantité de données totale des valeur de colonnes (en octet) =CVS X TNC		36 996 480 000 5 475 479 040 000
Column overhead	surcoût lié à la génération de colonnes CNS X TNC		850 919 040 000
Column Subtotal	Data+Column overhaed		6 326 398 080 000
Row header overhead	surcoût lié à la génération de lignes(octet)		318 580 800
Row bloom filter			18 631 838 400
Row index			3 558 732 518 400
Row Subtotal	total lignes		3 577 682 937 600
Data file subtotal	Column Subtotal+Row Subtotal		9 904 081 017 600
Per Node			297 122 430 528
SSTABLE index			184 982 400
SSTABLE bloom filter			19 269 003
Base Storage	Data file subtotal		9 904 285 269 003
Additional Replicas	réplicas additionnels		19 808 570 538 005
Compaction Overhead	surcoût additionnel dû au compactage		29 712 855 807 008
Total Storage	STOCKAGE TOTAL		59 425 711 614 015
Per Node	Total par neoud		594 257 116 140
Actual column value data	% pris par les données en colonnes		55,28%
Column overhead	% du surcoût de la génération de colonnes		8,59%
Row overhead	% du surcoût de la génération de lignes		36,12%
SSTABLE overhead	% surcoût additionnel dû au compactage		0,00%

Figure 10 : Dimensionnement du schéma, second test.

3.3 Processus de récupération des données

Le script Keyspace utilisé et inséré dans le module « CQL script » est le suivant, avec une simple stratégie (cela signifie que la communication GOSSIP sera optimisée pour un cluster avec un seul Data Center) et 3 répliquions :

CREATE KEYSPACE meteo

```
with replication = { 'class' : 'SimpleStrategy', 'replication_factor' : 3 };
```

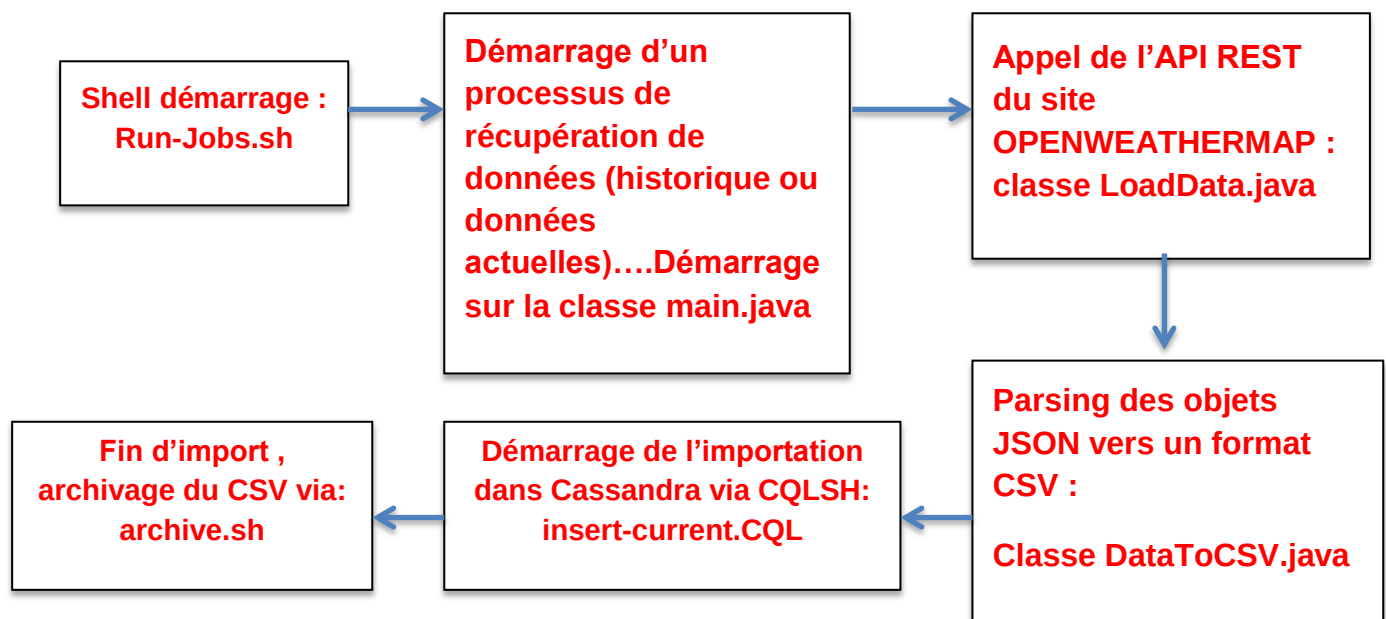
Ensuite nous avons ce script pour générer la famille de colonne :

CREATE TABLE weatherdata

```
(  
  partition      long, // intervalle de temps fixe en secondes (par exemple 1 jour ou un  
mois)  
  station_id int, // dans notre cas des stations météorologiques  
  collected_at timestamp, // date au format UNIX de la mesure  
  val blob, // données JSON collectées et stocké au format binaire.  
  PRIMARY KEY ((partition, station_id ), collected_at)  
);
```

Nous avons utilisé l'api cliente Java pour nous connecter à OpenWeatherMap. Nous avons généré un CSV avec une bibliothèque java afin d'utiliser la fonction COPY du shell Cassandra nommé CQLSH. Cette fonction prend un fichier CSV en paramètre et alimente en batch la famille de colonne adéquate. Nous avons pour résumer :

- Un jar pour récupérer les données via l'API REST de OpenWeatherMap ;
- Un script CQL démarré via le shell CQLSH ;
- Un script shell UNIX afin de lancer l'ensemble automatiquement.



CONCLUSION

La modélisation du risque de tempête est la modélisation d'un événement rare. Il s'agit dans le cas présent d'agréger ou combiner une série d'événements météorologiques qui, pris individuellement, ne provoquent pas de catastrophe naturelle.

Dans ce contexte, le SGBD devait être :

- un bon outil de collecte, afin d'atteindre une volumétrie significative.
- un outil qui permettait de stocker efficacement la donnée séquentiellement.
- un outil qui supportait des requêtes par lieu de collecte, par date, par valeur afin d'offrir une variété significative.
- un outil qui supportait l'ajout de variables sans risque de faire exploser l'architecture.

Cassandra présentant toutes ces caractéristiques, nous l'avons choisi.

De plus Cassandra est :

- très rapide en écriture.
- offre une gestion simplifiée des clusters grâce au support du clustering sur les nœuds virtuels.
- ce support du clustering sur les nœuds virtuels améliore les performances lors de l'ajout et de la reconstruction des nœuds.

Mais il conviendra de prendre en compte un certain nombre de faiblesses :

- Cassandra a été conçue par Facebook, et vite abandonnée par celui-ci à cause du high latency, ce "high latency" résulte de la nécessité de faire coïncider les données (obligation de cohérence).
- Afin d'éviter les collisions de données, il est recommandé d'utiliser Time UUID, un outil mis à la disposition des usagers de Cassandra pour éviter ces collisions.

Enfin, une utilisation optimale de Cassandra nécessite certaines précautions :

- Il conviendra ainsi d'éviter d'exécuter une requête en mode "raw cache" hors mémoire, ainsi que d'utiliser l'outil de compression.
- Il conviendra de "purger" les requêtes pendantes" (pending queues) en créant un pointer.
- Il conviendra d'utiliser en tant que de besoin l'outil « cfhistograms » recommandé par Cassandra qui permet de fournir des statistiques sur un tableau qui permet de tracer des courbes de fréquences.

Le choix du SGBD est important, mais ne suffit pas, la pertinence d'une telle modélisation repose aussi sur **la création de variables à partir des données collectées**, leur corrélation et la robustesse de l'algorithme.

TABLE DES FIGURES

Figure 1 : un exemple de communication multi data centers pour une demande d'écriture.....	4
Figure 2 : vue logique d'une ligne dans Cassandra (erratum: les lignes ne sont pas triées comme pourrait le suggérer la partie gauche de la figure)	5
Figure 3 : vue du mécanisme d'écriture dans Cassandra	7
Figure 4 : vue du mécanisme de lecture.....	9
Figure 5 : modèle de données classique pour les séries temporelles	11
Figure 6 : la différence entre la réplication par noeud de celle par noeud virtuel pour un facteur de réplication à 3	13
Figure 7: le théorème de CAP adapté aux systèmes analytiques temps-réels	14
Figure 8 : Echelle de Beaufort	16
Figure 9 : Dimensionnement du schéma, premier test.....	17
Figure 10 : Dimensionnement du schéma, second test	18

PRINCIPAUX LIENS INTERNET

www.datastax.com
<http://cassandra.apache.org/>
<http://planetcassandra.org/what-is-apache-cassandra/>
<http://soat.developpez.com/articles/cassandra/>
<http://techblog.netflix.com/2011/11/benchmarking-cassandra-scalability-on.html>
www.computer.org/intelligent
<http://www.larevuedudigital.com/2014/11/23/pour-reussir-il-faut-creer-de-laddiction-chez-son-client-grace-a-la-donnee-pour-francois-bourdoncle/>
www.CCR.fr
<http://www.axa.com/en/news/2015/parametric-insurance.aspx>
<http://www.lesechos.fr/finance-marches/banque-assurances/0204152662844-meteo-axa-se-developpe-dans-lassurance-indicielle-1092405.php>

BIBLIOGRAPHIE & CONFERENCES

11/2014 : Étude E&Y Big Data « Où en sont les entreprises françaises ».
05/2014 : Conférence Françoise Soulié (Terralab) « Utiliser les Big data : Défis & Opportunités »
10/2014 : Conférence Groupe de Travail Big Data de l'Institut des Actuaire « Données Internes – Données externes »
02/2015 : Étude « Boston Consulting Group » sur la transformation digitale des assureurs ».

ANNEXE – Les scripts

Les scripts

Ci-dessous les scripts pour la récupération des données actuelles, pour les historiques c'est à peu près la même chose.

Main.java :

```
package wind;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
import org.json.simple.parser.ParseException;

import wind.loadStationId;
public class Main {
    public static String FileName;
    protected static final String dir=System.getProperty("user.dir");
    static
    {
        FileName=dir.concat("/src/main/resources/city.list.json");
    }
    public static void main(String[] args) {

        if(args.length>0 )
        {
            if(args[0].equals("history"))
            {
                try {
                    ReadJSON(new File(FileName),"UTF-8");

                } catch (FileNotFoundException e) {
                    e.printStackTrace();
                } catch (ParseException e) {
                    e.printStackTrace();
                }
                catch (Exception e){
                    e.printStackTrace();
                }
            }
            if(args[0].equals("-city"))
            {
                try {
                    ReadCityJSON(new File(FileName),"UTF-8");

                } catch (FileNotFoundException e) {
                    e.printStackTrace();
                } catch (ParseException e) {
                    e.printStackTrace();
                }
                catch (Exception e){
                    e.printStackTrace();
                }
            }
        }
    }
}
```

```

        else{
            System.out.println("can't understand your arguments");
        }
    }
    else
    {

        try {

            loadCurrentData();
        }
        catch (Exception e){
            e.printStackTrace();
        }
    }
}

public static void ReadJSON(File MyFile,String Encoding) throws
FileNotFoundException, ParseException {
    Scanner scn=new Scanner(MyFile,Encoding);
    loadStationId LoadStationId= new loadStationId(scn);

}

public static void ReadCityJSON(File MyFile,String Encoding) throws
FileNotFoundException, ParseException {
    Scanner scn=new Scanner(MyFile,Encoding);
    LoadCityList cityList= new LoadCityList(scn);

}

public static void loadCurrentData()
{
    loadCurrentData lcd= new loadCurrentData();
}
}

```

loadCurrentData.java :

```

package wind;
import org.json.JSONException;
import java.io.IOException;
import java.util.Scanner;
import org.bitpipeline.lib.owm.WeatherStatusResponse;
import org.bitpipeline.lib.owm.WindClient;

```

```

public class loadCurrentData {
    public String id;
    public String name;
    public String country;
    public String city;
    public int Long;
    public int Lat;
    public static String FileName;
    public String stationId="0";
    //protected Session session;
    //protected ResultSet result;
    protected Scanner scn;
    //protected JSONObject obj;
    protected CassandraClient cassClient;
}

```

```

protected final String node="127.0.0.1";
int cpt=0;
WindClient client= new WindClient();
public WeatherStatusResponse response;
public CurrentDataToCSV wData;
protected cassandraBulkLoader bulkLoader;
public String pathToCSV;
private float eastLon=(float)36.59 ;
private float westLon=(float)67.24;
private float northLat=(float)-17.56;
private float southLat=(float)15.38;

public loadCurrentData()
{
    try{
        wData= new CurrentDataToCSV();
        this.readScan();
        pathToCSV=wData.closeMapWriter();
    }
    catch(Exception e){e.printStackTrace();}
}
public void readScan()
{
    try {
        response=client.currentWeatherAtCityBoundingBox(northLat,    westLon,    southLat,
        eastLon);

        wData.getProcessors();

        wData.writeWithCsvMapWriter(response);
    } catch (JSONException e) {

        e.printStackTrace();
    } catch (IOException e) {

        e.printStackTrace();
    }
    catch (Exception e) {

        e.printStackTrace();
    }

}
}

```

CurrentDataToCSV .java:

```

package wind;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.text.DateFormat;
import java.text.FieldPosition;

```

```

import java.text.SimpleDateFormat;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import org.bitpipeline.lib.owm.LocalizedWeatherData.GeoCoord;
import org.bitpipeline.lib.owm.WeatherData.WeatherCondition;
import org.bitpipeline.lib.owm.WeatherData.Clouds.CloudDescription;
import org.bitpipeline.lib.owm.StatusWeatherData;
import org.bitpipeline.lib.owm.WeatherStatusResponse;
import org.supercsv.cellprocessor.Optional;
import org.supercsv.cellprocessor.ift.CellProcessor;
import org.supercsv.io.CsvMapWriter;
import org.supercsv.io.ICsvMapWriter;
import org.supercsv.prefs.CsvPreference;
public class CurrentDataToCSV {
    public List<StatusWeatherData> IWDData;
    ICsvMapWriter mapWriter = null;
    protected static final String dir=System.getProperty("user.dir");
    protected String fileName;
    protected String archiveName;
    protected String shellCommand;
    protected static String pathToCSV;
    protected static String pathToCSVArchive;
    protected final DateFormat dateformat= new SimpleDateFormat("dd-MM-yyyy");
    protected CellProcessor[] processors; // CSV directory
    final String[] header = new String[] { "id", "dt", "name","date","lat","lon",
        "message","country","sunrise","sunset", "temp", "humidity",
        "temp_min", "temp_max","pressure","sea_level","grnd_level","speed",
        "deg","gust","clouds",
        "rain","snow","weather_code"};
    static{
        pathToCSV=dir.concat("/current/");
        pathToCSVArchive=dir.concat("/current/archive/");
    }
    /**
     * Sets up the processors used for the examples. There are 10 CSV columns, so 10
processors are defined. All values
     * are converted to Strings before writing (there's no need to convert them), and null
values will be written as
     * empty columns (no need to convert them to "").
     *
     * @return the cell processors
     */
    public void getProcessors() {

        processors = new CellProcessor[] {
            new Optional(), // Id 0
            new Optional(), // dt 1
            new Optional(), // name 2
            new Optional(), //Date 3
            new Optional(), // lat 4
            new Optional(), // lon 5
            new Optional(), //message 6
            new Optional(), // country 7
            new Optional(), // sunrise 8

```

```

        new Optional(), // sunset 9
        new Optional(), // temp 10
        new Optional(), // humidity 11
        new Optional(), // temp_min 12
        new Optional(), //temp_max 13
        new Optional(), //pressure 14
        new Optional(), //sea_level 15
        new Optional(), //grnd_level 16
        new Optional(), //speed 17
        new Optional(), //deg 18
        new Optional(), //gust 19
        new Optional(), //clouds 20
        new Optional(), //rain 21
        new Optional(), //snow 22
        new Optional(), //weather_code 23
    };

    try {
        shellCommand=pathToCSV+"batch-run-CQLSH.sh";
        fileName=pathToCSV+"current.csv";
        archiveName=pathToCSVArchive+dateFormat.format(new
java.util.Date(System.currentTimeMillis())).toString();
        mapWriter = new CsvMapWriter(new
FileWriter(fileName),CsvPreference.EXCEL_NORTH_EUROPE_PREFERENCE);

        // final CellProcessor[] processors = getProcessors();

        // write the header
        mapWriter.writeHeader(header);

        // write the customer maps
        // mapWriter.write(wRow, header, processors);

    }
    catch(Exception e){
        e.printStackTrace();
    }

}
/**
 * An example of reading using CsvMapWriter.
 */
public void writeWithCsvMapWriter(WeatherStatusResponse response) throws
Exception {

        IWData= response.getWeatherStatus();

        for(StatusWeatherData wData:IWData)
        {
            // create the weather row Maps (using the header elements for the
column keys)
            final Map<String, Object> wRow = new HashMap<String, Object>();

```

```

        GeoCoord coord=(GeoCoord)wData.getCoord();
        String wCond="";wData.getClouds().getAll();
        for(WeatherCondition wc:wData.getWeatherConditions())
        {
            wCond.concat(" "+wc.getDescription());
        }
        String wClouds="bla";
        for(CloudDescription wcl:wData.getClouds().getConditions())
        {
            wClouds.concat(" "+(wcl.getCumulus().getDescription()));
        }
        wRow.put(header[0], wData.getId());
        wRow.put(header[1], wData.getDateTime());
        wRow.put(header[2], wData.getName());
        wRow.put(header[3],
java.util.Date((long)wData.getDateTime()*1000),new           dateformat.format(new
FieldPosition(0)).toString());                               StringBuffer(),           new

        wRow.put(header[4], coord.getLatitude());
        wRow.put(header[5], coord.getLongitude());
        wRow.put(header[6], response.getMessage());
        wRow.put(header[7], "FR");
        // wRow.put(header[8], );
        // wRow.put(header[9], wData.getClouds().toString());
        wRow.put(header[10], wData.getTemp());
        wRow.put(header[11], wData.getHumidity());
        wRow.put(header[12], wData.getMain().getTempMin());
        wRow.put(header[13], wData.getMain().getTempMax());
        wRow.put(header[14], wData.getMain().getPressure());
        wRow.put(header[17], wData.getWind().getSpeed());
        wRow.put(header[18], wData.getWind().getDeg());
        wRow.put(header[19], wData.getWind().getGust());
        wRow.put(header[20], wData.getClouds().getAll());
        wRow.put(header[21], wData.getRain());
        wRow.put(header[22], wData.getSnow());
        wRow.put(header[23], wCond);

        try {

            //

            // write the header

            // write the customer maps
            mapWriter.write(wRow, header, processors);

        }
        finally {

        }

    }

}

public void archiveCSV()
{
    try{

```

```

        File source = new File(fileName);

        if(source.renameTo(new File(archiveName))){
            System.out.println("File is moved successfully!");
        }else{
            System.out.println("File is failed to move!");
        }

    }catch(Exception e){
        e.printStackTrace();
    }
}
public void runCommand()
{
    try {
        Runtime.getRuntime().exec(shellCommand);
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}
public String closeMapWriter()
{
    if( mapWriter != null )
    {
        try {
            mapWriter.close();
        }
        catch (IOException e)
        {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
    return fileName;
}
}

```

INSERT-CURRENT.CQL :

```

COPY meteo.weatherdata(partition,station_id,collected_at,val)
FROM 'PATH_TO_CURRENT_CSV/current.csv' WITH HEADER = true AND DELIMITER=';';

```

ANNEXE - Glossaire

Assurance non-vie : ou assurances de biens et de responsabilité (également appelée assurances dommages), compensent les pertes financières dues à la destruction de biens matériels ou aux dommages causés par un tiers ou par un évènement climatique.

Risque tempête : En France le régime des catastrophes naturelles (institué par la loi n°82-600 du 13 juillet 1982 et géré par la Caisse Centrale de Réassurance dite la CCR) produit une vaste couverture contre les dommages issus de catastrophes naturelles pour un prix modéré (ce prix est compris dans l'assurance multirisque habitation environ une vingtaine d'euros). Mais les périls concernés sont les inondations, les tremblements de terre, la sécheresse sur bâtiments (qu'on appelle également la subsidence) et les cyclones tropicaux. Les tempêtes sont exclues du périmètre. Aussi le risque « tempête » est laissé à la main des assureurs qui pour s'en prémunir font souvent appel à des programmes de réassurance.

Réassurance (ou l'assurance de l'assurance) : se traduit par des contrats (des traités ou appelés aussi des programmes de réassurance) passés entre l'assureur et son réassureur. La réassurance est l'un des outils de pilotage des plus importants des assureurs non-vie. C'est grâce à ce transfert de risque que l'assureur diminue ses sinistres et sa probabilité de ruine. Le risque de catastrophe naturelle est un risque qui a le potentiel de réduire significativement les fonds propres d'un assureur, voire de le mettre en faillite. Aussi les enjeux sont colossaux pour les assureurs non-vie et des outils performants et transparents sont nécessaires à l'identification de ses risques, leur exposition et le coût d'une destruction sur les biens de ses assurés.

Portefeuille synthétique (Principe basé sur les portefeuilles obligataires synthétiques qui remplace des valeurs des obligations par une combinaison de contrats à terme) : Les données des assureurs ne sont malheureusement pas « ouvertes » au public, aussi il sera créé un portefeuille fictif qui permettra de tester des algorithmes. Ce portefeuille sera en fait une base de données (.csv) comportant les caractéristiques des polices d'assurance avec pour citer les principales : le numéro (fictif) de la police, le secteur d'activité (résidentiel, commercial, industriel ou agricole), les sommes assurées (c'est-à-dire la valeur totale du bien), le code postal du lieu où se trouve le bien (localisation du bien), valeur du contenant (du bâtiment), la valeur du contenu etc.

Échelle de Beaufort : Francis Beaufort (1774-1857) a créé une échelle anémométrique (art de la mesure du vent, de sa vitesse, de sa direction, qui porte désormais son nom. Utilisée essentiellement par les marins et navigateurs, elle exprime la force du vent mesurée en degrés Beaufort, à une hauteur de 10 mètres au-dessus d'un terrain plat et découvert; en fait, il s'agit généralement d'un calcul effectué à partir de la vitesse moyenne du vent calculée sur une période de 10 minutes.